Київський національний університет імені Тараса Шевченка
Механіко-математичний факультет

Науково-практичний звіт з лабораторної роботи
з дисципліни «Обчислювальна геометрія»

Усі найближчі сусіди

Алгоритм «розділяй і володарюй» на основі
тріангуляції Делоне зі складністю $O(N \log N)$

Шифр завдання: АБ1, варіант № 46

Виконав:
студент 3 курсу
групи комп'ютерної математики
Батрак Кирило Володимирович

Викладач:
проф. Терещенко Василь Миколайович

Київ — 2026

Зміст

Анотація **3**

Abstract **3**

1 Вступ **4**

1.1 Проблематика та актуальність 4

1.2 Аналіз існуючих підходів 4

1.3 Новизна та ідея запропонованого підходу 5

1.4 Мета роботи 5

2 Основна частина **6**

2.1 Постановка задачі 6

2.2 Основні означення та твердження 6

2.3 Аргументація обраного методу 8

2.4 Опис алгоритму 8

3 Обґрунтування оцінки складності **11**

3.1 Покрокова оцінка 12

3.2 Підсумкова оцінка 13

3.3 Експериментальне підтвердження 14

4 Практична частина **14**

4.1 Особливості програмної реалізації 14

4.2 Опис основних функцій 16

4.3 Характеризація вводу та виводу даних 16

4.4 Можливості, програмне та технічне забезпечення 18

4.5 Демонстрація на великих і найгірших вхідних даних 19

4.6 Лістинг основних модулів 20

5 Висновки **21**

Список літератури **24**

Додатки **26**

А Модуль `delaunay.py` **26**

Б Модуль `nearest.py` **28**

Анотація

У роботі розглянуто задачу «Усі найближчі сусіди» для множини з N точок евклідової площини. Задачу зведено до побудови тріангуляції Делоне, яку будує рекурсивний алгоритм «розділяй і володарюй» з точними цілочисельними предикатами, після чого найближчого сусіда кожної точки знаходимо за $O(N)$ переглядом інцидентних їй ребер; загальна складність становить $O(N \log N)$, що збігається з нижньою оцінкою задачі. Реалізацію перевірено на 156 автоматичних тестах і порівнянням із повним перебором, а заміри для N до $1{,}6 \cdot 10^4$ підтверджують теоретичну оцінку, зокрема на вироджених вхідних даних.

Abstract

The paper considers the All Nearest Neighbors problem for a set of N points in the Euclidean plane. The problem is reduced to constructing the Delaunay triangulation with a recursive divide-and-conquer algorithm based on exact integer predicates, after which the nearest neighbor of every point is found in $O(N)$ time by scanning its incident edges; the overall complexity is $O(N \log N)$, which matches the lower bound for the problem. The implementation is verified by 156 automated tests and by comparison with brute force, and measurements for N up to $1.6 \cdot 10^4$ confirm the theoretical bound, including on degenerate inputs.

1. Вступ

1.1. Проблематика та актуальність

Задача «Усі найближчі сусіди» (All Nearest Neighbors, далі ANN) належить до класу задач близькості обчислювальної геометрії [11, 12, 13]. Для кожної точки заданої множини потрібно знайти найближчу до неї іншу точку цієї ж множини. Задача виглядає простою, але її наївний розв'язок перебирає всі $N(N-1)/2$ пар точок і вже для $N = 10^4$ потребує близько $5 \cdot 10^7$ обчислень відстані. У прикладних задачах точок буває 10^6 і більше, тому квадратичний перебір там непридатний.

Граф найближчих сусідів, тобто сукупність пар $p \to \mathrm{NN}(p)$, є проміжним кроком у багатьох алгоритмах:

- кластерний аналіз (ієрархічна кластеризація за одиничним зв'язком, оцінка локальної щільності, пошук аномальних точок);
- геоінформаційні системи (найближчий об'єкт інфраструктури для кожного об'єкта мапи);
- обчислювальна фізика та астрофізика, де для кожної частинки чи галактики потрібні сусіди в моделях на мільйони точок [8, 10];
- побудова сіток і обробка хмар точок у комп'ютерній графіці та комп'ютерному зорі [9].

Відомо, що задача НАЙБЛИЖЧА ПАРА за $O(N)$ зводиться до ANN, а в моделі алгебраїчних дерев обчислень будь-який алгоритм для ANN потребує $\Omega(N \log N)$ операцій [13] (теореми 6.4 і 6.6). Тому алгоритм зі складністю $O(N \log N)$ є оптимальним за порядком, і саме таку оцінку вимагає завдання.

1.2. Аналіз існуючих підходів

Повний перебір. Обчислює відстані між усіма парами за $\Theta(N^2)$. Він простий і безпомилковий, тому в роботі використовується як еталон для перевірки коректності.

Через діаграму Вороного. Класичний підхід Шеймоса–Гоуі будує діаграму Вороного $\mathrm{Vor}(S)$ методом «розділяй і володарюй» за $O(N \log N)$: множину ділять навпіл, рекурсивно будують діаграми половин і зшивають їх розділяючим ланцюгом σ [11, 13]. Найближчий сусід кожної точки визначає ребро її многокутника Вороного, тому за наслідком теореми 12 з лекції 10 після побудови $\mathrm{Vor}(S)$ задача ANN розв'язується за $\Theta(N)$ [13]. Недолік підходу в складному кроці злиття: ланцюг σ потребує обчислення перетинів серединних перпендикулярів у дійсних числах, що чутливо до похибок округлення та вироджених конфігурацій.

Через тріангуляцію Делоне. Тріангуляція Делоне $\mathrm{DT}(S)$ є графом, двоїстим до $\mathrm{Vor}(S)$ [13]. У книзі Препарати та Шеймоса зазначено, що її можна будувати безпосередньо рекурсивною процедурою, аналогічною процедурі для діаграми Вороного, за оптимальний час (метод Лі–Шахтера) [11]. Найбільш відомий варіант цього методу належить Гібасу та Столфі [14]: він зберігає тріангуляцію у структурі quad-edge і зводить усю геометрію до двох предикатів, орієнтації трійки точок та

перевірки «точка всередині кола».

Рандомізовані та прості алгоритми. Для ANN на площині відомі рандомізовані алгоритми з очікуваним часом $O(N \log N)$, які не будують жодної планарної структури [1]. Споріднену задачу про найближчу пару продовжують досліджувати: будують динамічні структури даних, що підтримують пошук найближчого сусіда та найближчої пари при додаванні й видаленні точок [2], будують структури для запитів найближчої пари в діапазоні [3] та отримують точні оцінки кількості найближчих пар у вертикальних смугах [4]. Робота [5] показує, що найближча пара в просторах вищої розмірності розв'язується майже за лінійний час, тоді як близькі за формулюванням задачі (найвіддаленіша пара, ортогональні вектори) лише трохи швидші за квадратичний перебір.

Просторові дерева та паралельні методи. На практиці для пошуку сусідів широко використовують k-d-дерева та подібні структури. Сучасні роботи присвячені їхнім паралельним версіям із пакетним оновленням [6, 7]. Діаграми Вороного великих множин будують паралельно в розподіленій пам'яті [8] та на графічних процесорах [9, 10]. Ці методи досягають високої продуктивності, але гарантії складності для найгіршого вводу в них слабші, ніж у детермінованих геометричних алгоритмах.

1.3. Новизна та ідея запропонованого підходу

У роботі розв'язок задачі ANN побудовано як модифікацію методу Гібаса–Столфі, пристосовану саме до пошуку сусідів:

1. Діаграма Вороного явно не будується. Задача зводиться одразу до тріангуляції Делоне: доведено (теорема 1), що ребро до *кожного* найближчого сусіда, зокрема до всіх рівновіддалених, належить *будь-якій* тріангуляції Делоне. Через це результат не залежить від вибору тріангуляції у вироджених випадках.
2. Усі геометричні перевірки виконуються в точній цілочисельній арифметиці. Злиття половин використовує лише знаки двох визначників і не обчислює жодного перетину, тому алгоритм коректний для колінеарних, коциклічних і повторюваних точок без окремих обхідних шляхів.
3. Результат детермінований і повністю збігається з повним перебором, включно з вибором сусіда серед рівновіддалених (з меншим індексом). Це дозволяє перевіряти алгоритм порівнянням індексів, а не лише відстаней.

Вибір цього методу замість побудови ланцюга σ з лекції 10 погоджено з викладачем.

1.4. Мета роботи

Мета роботи: для множини S з N точок площини розробити та реалізувати алгоритм «розділяй і володарюй», який за час $O(N \log N)$ знаходить найближчого сусіда кожної точки; обґрунтувати його коректність і складність; експериментально перевірити правильність і швидкодію, зокрема на найгірших вхідних даних; створити програму з ручним вводом до 100 точок, автоматичною генерацією порядку 10^4 точок і наочною

візуалізацією результату.

2. Основна частина

2.1. Постановка задачі

Нехай задано множину $S = \{p_0, p_1, \ldots, p_{N-1}\}$ з $N \geq 2$ точок евклідової площини E^2, $p_i = (x_i, y_i)$. Відстань між точками

$$d(p, q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}.$$

Найближчим сусідом точки p_i називається точка $p_j \in S$, $j \neq i$, для якої

$$d(p_i, p_j) = \min_{k \neq i} d(p_i, p_k).$$

Потрібно для кожного $i = 0, \ldots, N-1$ знайти індекс $\mathrm{NN}(i)$ найближчого сусіда точки p_i. Якщо рівновіддалених сусідів декілька, обирається сусід з найменшим індексом, тож відповідь визначена однозначно. Точки з однаковими координатами вважаються сусідами одна одної на відстані 0. Метод розв'язання: «розділяй і володарюй», складність $O(N \log N)$.

Зауважимо, що відношення «бути найближчим сусідом» несиметричне: з $\mathrm{NN}(p) = q$ не випливає $\mathrm{NN}(q) = p$ (рис. 3 б). Тому задачу не можна звести до пошуку однієї найближчої пари. Пари з $\mathrm{NN}(p) = q$ і $\mathrm{NN}(q) = p$ далі називатимемо *взаємними*.

2.2. Основні означення та твердження

Означення 1 (многокутник і діаграма Вороного)**.** Многокутником Вороного точки $p_i \in S$ називається множина $V(i) = \{x \in E^2 : \ d(x, p_i) \leq d(x, p_j)\ \forall j \neq i\}$. Розбиття площини на многокутники $V(0), \ldots, V(N-1)$ називається діаграмою Вороного $\mathrm{Vor}(S)$ [13].

Означення 2 (тріангуляція Делоне)**.** Тріангуляція множини S, обмежена границею опуклої оболонки $CH(S)$, називається тріангуляцією Делоне $\mathrm{DT}(S)$, якщо кола, описані навколо всіх її трикутників, не містять усередині жодної точки множини S (*умова Делоне*) [13].

Використовуватимемо такі відомі властивості [13]:

(B1) $\mathrm{DT}(S)$ є графом, двоїстим до $\mathrm{Vor}(S)$: точки p, q з'єднані ребром $\mathrm{DT}(S)$ тоді й лише тоді, коли многокутники $V(p)$ і $V(q)$ мають спільне ребро.

(B2) Якщо існує коло, що проходить через дві точки $p, q \in S$ і не містить усередині та на собі жодних інших точок S, то pq є ребром $\mathrm{DT}(S)$.

(B3) Діаграма Вороного N точок має не більше $2N - 5$ вершин і $3N - 6$ ребер; відповідно, $\mathrm{DT}(S)$ має не більше $3N - 6$ ребер і $2N - 5$ трикутників.

Властивість (B3) випливає з формули Ейлера $V - E + F = 2$ для планарного графа: у тріангуляції кожна внутрішня грань обмежена трьома ребрами, кожне ребро

межує з двома гранями, звідки $E \leq 3V - 6$.

Теорема 1. *Нехай q є найближчим сусідом точки p у множині S (не обов'язково єдиним). Тоді відрізок pq є ребром будь-якої тріангуляції Делоне множини S.*

Доведення. Розглянемо круг K з діаметром pq (рис. 1). Нехай точка $r \in S$, $r \neq p, q$, лежить у K, усередині або на границі. Кут $\angle prq$, що спирається на діаметр, не менший за $\pi/2$, тому в трикутнику prq сторона pq лежить проти найбільшого кута і $d(p, r) < d(p, q)$. Отже r була б ближчою до p, ніж q, що суперечить вибору q. Тому в K, крім p і q, немає точок множини S ні всередині, ні на границі. Коло, що обмежує K, проходить через p і q та задовольняє умову властивості (B2), тож $pq \in \mathrm{DT}(S)$.

Оскільки на границі K немає інших точок S, ребро pq не може бути «діагоналлю», яку вибирають довільно серед кількох коциклічних точок. Отже, pq належить кожній тріангуляції Делоне, а не лише одній із них. □

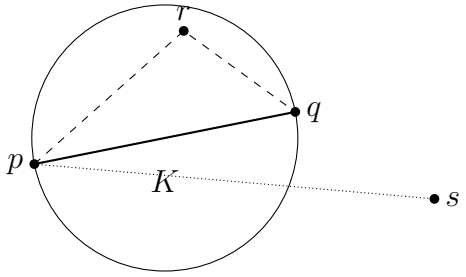


Рис. 1. До доведення теореми 1: будь-яка точка r круга з діаметром pq ближча до p, ніж q; точки, віддаленіші за q (наприклад, s), лежать поза кругом

Наслідок 1. *Для кожної точки p найближчий сусід міститься серед її сусідів у* $\mathrm{DT}(S)$. *Сумарна кількість таких сусідів дорівнює $\sum_p \deg(p) = 2E \leq 6N - 12$, тому після побудови* $\mathrm{DT}(S)$ *задача ANN розв'язується за $O(N)$.*

Наслідок 1 є версією наслідку теореми 12 з лекції 10 [13], сформульованою для двоїстого графа, і узгоджується з відомою властивістю тріангуляції Делоне: граф найближчих сусідів є підграфом $\mathrm{DT}(S)$ [13].

Геометричні предикати

Для точок з цілими координатами алгоритм використовує лише два предикати, які обчислюються точно:

$$\mathrm{ccw}(a, b, c) = \begin{vmatrix} x_b - x_a & y_b - y_a \\ x_c - x_a & y_c - y_a \end{vmatrix}, \tag{1}$$

$$\mathrm{in_circle}(a, b, c, d) = \begin{vmatrix} x_a - x_d & y_a - y_d & (x_a - x_d)^2 + (y_a - y_d)^2 \\ x_b - x_d & y_b - y_d & (x_b - x_d)^2 + (y_b - y_d)^2 \\ x_c - x_d & y_c - y_d & (x_c - x_d)^2 + (y_c - y_d)^2 \end{vmatrix}. \tag{2}$$

Знак $\mathrm{ccw}(a, b, c)$ визначає орієнтацію трійки: додатний для лівого повороту (проти годинникової стрілки), від'ємний для правого, нуль для колінеарних точок. Для a, b, c з

$\mathrm{ccw}(a,b,c) > 0$ знак $\mathrm{in_circle}(a,b,c,d)$ додатний, якщо d лежить строго всередині кола, описаного навколо abc, від'ємний, якщо зовні, і дорівнює нулю для коциклічних точок. Відстані порівнюються через квадрати $d^2(p,q)$, бо $\sqrt{t}$ монотонна, тому добування кореня не потрібне.

Структура даних quad-edge

Тріангуляцію зберігаємо у структурі quad-edge Гібаса–Столфі. Кожне неорієнтоване ребро подано чотирма орієнтованими «чверть-ребрами» e, $e.\mathrm{rot}$, $e.\mathrm{sym}$, $e.\mathrm{rot}^{-1}$: два з них описують ребро тріангуляції в обох напрямках, два інші є двоїстим ребром діаграми Вороного. Для кожного чверть-ребра зберігаються лише початкова вершина org, посилання onext (наступне ребро з тим самим початком проти годинникової стрілки) та rot. Усі інші переходи (sym, lnext, oprev тощо) обчислюються через них за $O(1)$. Структуру змінюють три операції: `make_edge` (нове ізольоване ребро), `splice` (злиття або розрізання кілець onext) та похідні від них `connect` і `delete_edge`; кожна виконується за $O(1)$. Таким чином, одна структура містить і $\mathrm{DT}(S)$, і $\mathrm{Vor}(S)$, що відповідає властивості (B1).

2.3. Аргументація обраного методу

Пряме перенесення алгоритму пошуку найближчої пари з лекції 9 на задачу ANN не працює. У найближчій парі на етапі злиття перевіряється смуга ширини 2δ навколо лінії поділу, де δ є мінімальною відстанню в половинах. Для ANN кожна точка має власну відстань до сусіда, і для точки з великою такою відстанню сусід з іншої половини може лежати далеко від лінії поділу. Смуги ширини 2δ тут недостатньо, а перевірка всіх «далеких» точок дає квадратичний час у найгіршому випадку.

Тому, згідно з теоремою 12 і її наслідком з лекції 10, задачу ANN доцільно розв'язувати через діаграму Вороного або двоїсту до неї тріангуляцію Делоне. Обидва варіанти мають однакову асимптотику $O(N \log N)$, але для реалізації обрано тріангуляцію з таких причин:

- злиття тріангуляцій використовує лише знаки визначників (1) і (2), тоді як побудова ланцюга σ потребує обчислення точок перетину серединних перпендикулярів;
- для цілих координат обидва предикати обчислюються точно, тож алгоритм не має похибок округлення;
- з теореми 1 відповідь виходить одразу з тріангуляції, а діаграма Вороного, якщо вона потрібна для візуалізації, будується з тріангуляції за $O(N)$ як двоїстий граф.

2.4. Опис алгоритму

Алгоритм складається з чотирьох етапів (алгоритм 1).

Алгоритм 1 Усі найближчі сусіди

Вхід: точки $p_0, \dots, p_{N-1}$ з цілими координатами
Вихід: масив $\mathrm{NN}[0..N-1]$

1: згрупувати індекси за координатами: $G[p]$ — список індексів точки p за зростанням ▷ $O(N)$
2: **для всіх** груп $G[p]$ з $|G[p]| \geq 2$ **виконати** ▷ дублікати
3: кожному індексу групи присвоїти сусіда — найменший інший індекс групи
4: **кінець циклу**
5: $U \leftarrow$ різні точки; $\mathrm{rep}[p] \leftarrow \min G[p]$
6: $U \leftarrow$ сортувати за (x, y) ▷ $O(N \log N)$
7: $e \leftarrow$ Делоне$(U, 0, |U|)$ ▷ $O(N \log N)$
8: **для всіх** ребер ab тріангуляції **виконати** ▷ $O(N)$
9: оновити $\mathrm{best}[a]$ парою $(d^2(a,b), \mathrm{rep}[b])$; так само $\mathrm{best}[b]$
10: **кінець циклу**
11: **для всіх** точок p без дублікатів **виконати**
12: $\mathrm{NN}[\mathrm{rep}[p]] \leftarrow$ індекс з $\mathrm{best}[p]$
13: **кінець циклу**
14: **повернути** NN

Пари $(d^2, \text{індекс})$ порівнюються лексикографічно, тому серед рівновіддалених сусідів залишається сусід з меншим індексом. За теоремою 1 ребра до всіх рівновіддалених сусідів присутні в тріангуляції, тож результат збігається з повним перебором.

Побудова тріангуляції Делоне «розділяй і володарюй»

Функція Делоне (алгоритм 2) отримує точки, відсортовані за (x, y), і повертає два ребра опуклої оболонки: ldo (виходить з найлівішої точки) та rdo (з найправішої).

База рекурсії. Для двох точок створюється одне ребро. Для трьох точок створюються два ребра, а третє додається лише тоді, коли точки не колінеарні; орієнтація трикутника визначається знаком ccw.

Розділення. Відсортована множина ділиться за медіаною на ліву L і праву R половини, які розділені вертикальною прямою (при рівних x розділяє порядок за y). Для них рекурсивно будуються $\mathrm{DT}(L)$ і $\mathrm{DT}(R)$.

Злиття, крок 1: нижня спільна дотична. Починаючи з найправішої точки L і найлівішої точки R, рухаємося вниз по опуклих оболонках: поки точка R лежить справа від поточного ребра оболонки L, просуваємось по оболонці L, і навпаки. Отримуємо нижню спільну дотичну, яка стає першим ребром *базової лінії* basel між половинами. Це аналог нижнього опорного відрізка з алгоритму злиття діаграм Вороного в лекції 10.

Злиття, крок 2: «застібка». Далі половини зшиваються знизу вгору (рис. 2). На кожному кроці розглядаються два кандидати: lcand, ребро L, що виходить з лівого кінця basel, та rcand, аналогічне ребро R. Кандидат придатний, якщо його кінець лежить над basel. Поки коло через basel і кінець кандидата містить кінець наступного ребра віяла, ребро-кандидат порушує умову Делоне в новій тріангуляції і видаляється. З двох придатних кандидатів обирається той, чиє коло не містить кінця іншого; до

нього проводиться нове перехресне ребро, яке стає новою basel. Процес закінчується, коли обидва кандидати непридатні, тобто досягнуто верхньої спільної дотичної.

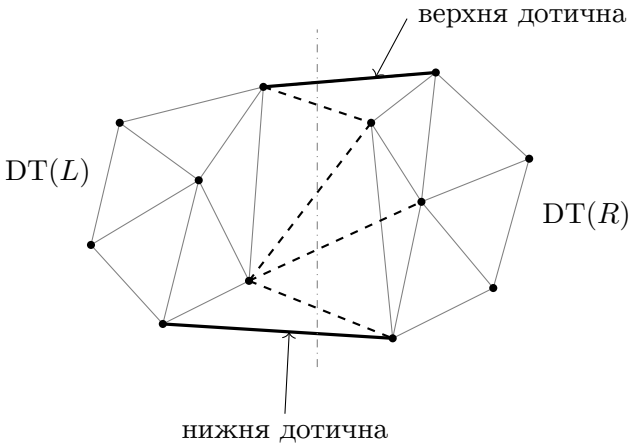


Рис. 2. Схема злиття: суцільні сірі ребра належать тріангуляціям половин, жирні є спільними дотичними, штрихові є перехресними ребрами, що додає «застібка» знизу вгору (схематично)

Алгоритм 2 Злиття тріангуляцій DT(L) і DT(R) [14]

(ldi, rdi) ← ребра оболонок біля лінії поділу
цикл ▷ нижня спільна дотична
    якщо rdi.org строго зліва від ldi **то** ldi ← ldi.lnext
    інакше якщо ldi.org строго справа від rdi **то** rdi ← rdi.rprev
    інакше вийти з циклу
    кінець умови
кінець циклу
basel ← Connect(rdi.sym, ldi)
цикл ▷ «застібка»
    lcand ← basel.sym.onext
    якщо lcand над basel **то**
        поки коло через basel і lcand.dest містить lcand.onext.dest **виконати**
            видалити lcand; lcand ← наступне ребро віяла
        кінець циклу
    кінець умови
    симетрично для rcand ← basel.oprev
    якщо жоден кандидат не лежить над basel **то вийти з циклу**
    кінець умови
    якщо lcand непридатний **або** коло lcand містить кінець rcand **то**
        basel ← Connect(rcand, basel.sym)
    інакше
        basel ← Connect(basel.sym, lcand.sym)
    кінець умови
кінець циклу

Вироджені випадки

- *Дублікати* відокремлюються до побудови тріангуляції за $O(N)$ хешуванням: копії однієї точки є сусідами одна одної на відстані 0, а в тріангуляцію по-

трапляє лише один представник.

- *Колінеарні точки.* Якщо всі точки лежать на одній прямій, рекурсія утворює ланцюжок ребер без трикутників (рис. 7б). Предикат $\mathrm{ccw} = 0$ коректно обробляється в базі рекурсії та при пошуку дотичної.
- *Коциклічні точки* (наприклад, решітка): $\mathrm{in_circle} = 0$ не призводить до видалення ребра, тож вибирається одна з допустимих тріангуляцій. За теоремою 1 відповідь від цього вибору не залежить.
- $N = 1$: сусіда не існує, результат $\mathrm{NN}(0) = \varnothing$.

Ілюстрація роботи

На рис. 3 показано результат для 20 точок, введених мишкою: тріангуляцію Делоне разом із двоїстою їй діаграмою Вороного та знайдені пари $p \to \mathrm{NN}(p)$. Видно, що кожна стрілка йде вздовж ребра тріангуляції, а п'ять взаємних пар позначено окремо.



(а) тріангуляція Делоне і діаграма Вороного (б) стрілки $p \to \mathrm{NN}(p)$ і взаємні пари

Рис. 3. Ручний ввід, $N = 20$: тріангуляція містить 51 ребро і 32 трикутники; 5 взаємних пар

3. Обґрунтування оцінки складності

Позначимо через N кількість вхідних точок, через $n \le N$ кількість різних точок. Модель обчислень: кожна арифметична операція над цілими числами, обчислення предиката (1) чи (2), операція над хеш-таблицею та кожна операція над quad-edge (`make_edge`, `splice`, `connect`, `delete_edge`, перехід по посиланню) виконуються за $O(1)$.

3.1. Покрокова оцінка

Крок 1. Дублікати. Один прохід по N точках з додаванням індексу до списку в хеш-таблиці: N операцій, тобто $O(N)$. Присвоєння сусідів у групах дублікатів зачіпає кожен індекс один раз: $O(N)$.

Крок 2. Сортування. n різних точок сортуються за ключем (x, y) порівняльним сортуванням: $O(n \log n)$. Сортування виконується один раз до рекурсії, тому в рекурсії поділ навпіл відбувається за $O(1)$ (обчислення середнього індексу).

Крок 3. Рекурсивна побудова DT**.** Нехай $T(n)$ є часом функції Делоне для n точок, $M(n)$ є часом злиття. Тоді

$$T(n) = \begin{cases} c_0, & n \le 3, \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + M(n), & n > 3. \end{cases} \tag{3}$$

База займає сталий час: для $n = 2$ створюється одне ребро, для $n = 3$ створюються два-три ребра і обчислюється один предикат.

Оцінка злиття $M(n)$**.** Нехай $|L| = n_L$, $|R| = n_R$, $n_L + n_R = n$.

1. *Нижня спільна дотична.* На кожній ітерації циклу вказівник ldi або rdi просувається на одне ребро своєї опуклої оболонки й ніколи не повертається назад. Оболонки мають не більше n_L і n_R вершин, тому ітерацій не більше $n_L + n_R = n$, і кожна коштує $O(1)$: $O(n)$.
2. *Видалення ребер у «застібці».* Кожна ітерація внутрішніх циклів видаляє одне ребро з $\mathrm{DT}(L)$ або $\mathrm{DT}(R)$. Видалене ребро вже ніколи не з'являється знову, а за властивістю (B3) ці тріангуляції разом мають не більше $(3n_L - 6) + (3n_R - 6) < 3n$ ребер. Отже, видалень менше за $3n$, кожне з одним обчисленням in_circle та $O(1)$ операцій з quad-edge.
3. *Додавання перехресних ребер.* Кожна ітерація зовнішнього циклу, крім останньої, додає одне перехресне ребро, яке залишається в результаті. Усі вони є ребрами $\mathrm{DT}(L \cup R)$, тому їх не більше $3n - 6$. Кожна ітерація виконує сталу кількість предикатів (не більше двох перевірок придатності, однієї перевірки in_circle для вибору кандидата) і одну операцію `connect`.

Сумарно

$$M(n) \le c_1 n + c_2 \cdot 3n + c_3 \cdot 3n = c\, n, \qquad \text{тобто } M(n) = O(n).$$

Розв'язок рекурентності. Підставимо $M(n) \le cn$ у (3) і вважатимемо для простоти $n = 2^k$. Розгорнемо рекурсію:

$$\begin{aligned} T(n) &\le 2T(n/2) + cn \\ &\le 2\big(2T(n/4) + cn/2\big) + cn = 4T(n/4) + 2cn \\ &\le \dots \\ &\le 2^j\, T(n/2^j) + j\, cn. \end{aligned}$$

Рекурсія зупиняється, коли $n/2^j \le 3$, тобто при $j = \lceil \log_2(n/3) \rceil$. Тоді

$$T(n) \le 2^j c_0 + cn \log_2 n \le n\, c_0 + cn \log_2 n = O(n \log n),$$

бо $2^j < 2n/3 \le n$. Для довільного n оцінка зберігається, бо $T(n)$ монотонна і $n \le 2^{\lceil \log_2 n \rceil} < 2n$. Те саме дає основна теорема про рекурентні співвідношення для $a = b = 2$, $f(n) = \Theta(n)$. Глибина рекурсії $\lceil \log_2 n \rceil$, тож для $n = 2 \cdot 10^4$ вона не перевищує 15 рівнів, і стек не переповнюється.

Крок 4. Пошук сусідів. Обхід усіх ребер тріангуляції функцією `iter_edges` відвідує кожне ребро один раз (множина відвіданих зберігається в хеш-таблиці), тобто $E \le 3n - 6$ кроків. Для кожного ребра ab оновлюються дві пари best$[a]$ і best$[b]$, разом $2E \le 6n - 12$ порівнянь (наслідок 1). Запис відповіді для n точок займає $O(n)$. Разом $O(n)$.

3.2. Підсумкова оцінка

Таблиця 1. Складність етапів алгоритму

Етап	Обґрунтування	Час
1. Дублікати	один прохід із хеш-таблицею	$O(N)$
2. Сортування	порівняльне сортування	$O(N \log N)$
3. Тріангуляція Делоне	$T(n) = 2T(n/2) + O(n)$	$O(N \log N)$
4. Пошук сусідів	$2E \le 6N - 12$ порівнянь	$O(N)$
Разом		$O(N \log N)$

Оскільки $n \le N$, загальний час становить

$$T_{\mathrm{ANN}}(N) = O(N) + O(N \log N) + O(N \log N) + O(N) = O(N \log N).$$

За теоремою 6.6 з лекції 9 будь-який алгоритм для задачі УСІ НАЙБЛИЖЧІ СУСІДИ в моделі дерев обчислень потребує $\Omega(N \log N)$ операцій [13]. Отже, побудований алгоритм оптимальний за порядком: $T_{\mathrm{ANN}}(N) = \Theta(N \log N)$ у найгіршому випадку.

Пам'ять. Quad-edge містить $4E \le 12n$ чверть-ребер, кожне з трьома полями; хеш-таблиці та масиви мають розмір $O(N)$; стек рекурсії має глибину $O(\log N)$. Отже, пам'ять $O(N)$.

Розрядність. Координати лежать у $[0, 10^6]$. Значення in_circle має порядок $(10^6)^4 = 10^{24}$, що перевищує межу 64-бітних цілих ($\approx 9{,}2 \cdot 10^{18}$). У Python цілі числа мають довільну точність, тому обчислення точне. Довжина операндів обмежена сталою, і кожен предикат залишається операцією $O(1)$. У мові з фіксованою розрядністю тут знадобилися б 128-бітні цілі.

3.3. Експериментальне підтвердження

Для перевірки оцінки виміряно час роботи на шести розподілах точок (рівномірний, нормальний, кластери, решітка, точки на одній прямій, точки біля кола) для N від 500 до 16 000. Повний перебір вимірювався лише на рівномірному розподілі до $N = 4000$, бо далі він стає надто повільним.

Таблиця 2. Час роботи на рівномірному розподілі, с

N	«розділяй і володарюй»	повний перебір	прискорення
500	0,019	0,032	1,7
1 000	0,041	0,129	3,2
2 000	0,089	0,523	5,9
4 000	0,187	2,144	11,5
8 000	0,398	—	—
16 000	0,852	—	—

У логарифмічних шкалах (рис. 4 а) залежність часу від N близька до прямої, і її нахил дорівнює показнику степеня. Для повного перебору нахил 2,03, що відповідає $\Theta(N^2)$. Для запропонованого алгоритму нахил 1,10; це трохи більше за 1 якраз за рахунок множника $\log N$. Нормований час $T(N)/(N \log_2 N)$ (рис. 4 б) для кожного розподілу майже сталий: на рівномірному розподілі він зменшується від 4,26 до 3,81 мкс при збільшенні N у 32 рази. Це емпірично підтверджує оцінку $O(N \log N)$. Вироджені розподіли не сповільнюють алгоритм: на решітці, прямій і колі нормований час навіть менший, ніж на рівномірному розподілі.

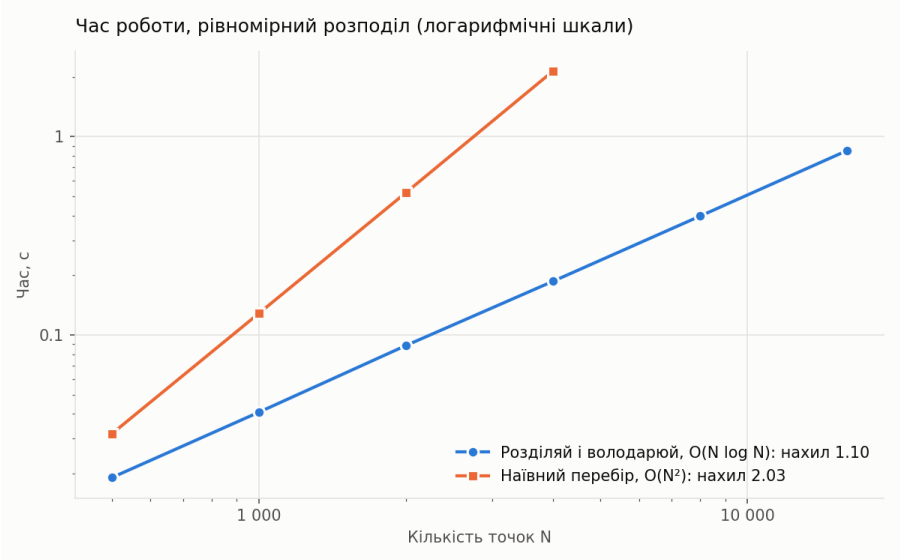
4. Практична частина

4.1. Особливості програмної реалізації

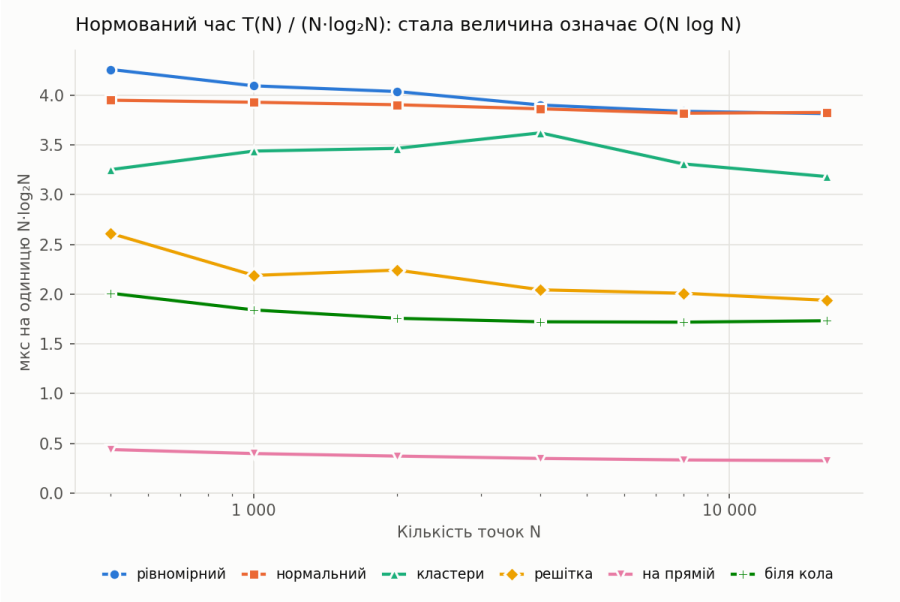
Програму написано мовою Python 3.13. Вихідний код розміщено в репозиторії `https://github.com/kyrylobatrak/all-nearest-neighbors` і поділено на три частини: обчислювальне ядро (`src/`), графічний інтерфейс (`ui/`) та автоматичні тести й заміри (`tests/`, `benchmarks/`). Ядро не залежить від інтерфейсу, тому той самий код використовують і програма, і тести, і заміри часу.

Основні рішення реалізації:

- *Точна арифметика.* Координати точок цілі в діапазоні $[0, 10^6]$, а предикати (1) і (2) обчислюються в цілих числах Python довільної точності. Похибки округлення, через які класичні реалізації тріангуляції «зациклюються» або будують неплоский граф на вироджених даних, тут виключені. Дійсні числа використовуються лише для малювання діаграми Вороного.
- *Структура quad-edge* реалізована класом `Edge` з полями `org`, `next`, `rot`; атрибут `__slots__` зменшує витрати пам'яті на кожне з $12N$ чверть-ребер.
- *Детермінованість.* Результат не залежить від порядку обходу: серед рівновіддалених сусідів завжди обирається менший індекс. Тому відповідь можна



(а) порівняння з повним перебором



(б) нормований час для шести розподілів

Рис. 4. Результати замірів часу роботи

порівнювати з повним перебором поелементно.

- *Фонові обчислення.* В інтерфейсі алгоритм запускається в окремому потоці (`QThread`), тож вікно не зависає на $2 \cdot 10^4$ точках. Застарілі результати (якщо користувач змінив точки під час обчислення) відкидаються.
- *Тестування.* Коректність перевіряють 156 автоматичних тестів `pytest`. Вони охоплюють предикати, quad-edge, умову Делоне, формулу Ейлера, опуклість оболонки, вироджені випадки ($N = 0, 1, 2$, дублікати, колінеарні та коциклічні точки, решітки) та звірку з повним перебором на понад 500 випадкових наборах. Окремо перевірено ще 3000 випадкових наборів поза тестами; розбіжностей не виявлено.

4.2. Опис основних функцій

Основні функції обчислювального ядра, візуалізації та генерації даних наведено в таблиці 3 разом з їхньою складністю. Функції з префіксом «_» є внутрішніми.

4.3. Характеризація вводу та виводу даних

Ввід. Програма підтримує три способи задати точки:

1. *Ручний ввід* до 100 точок: ліва кнопка миші додає точку, права видаляє найближчу; точку також можна ввести координатами x, y. Є кнопки «Скасувати» (видалити останню) і «Очистити».
2. *Автоматична генерація* від 2 до 20 000 точок. Користувач обирає розподіл (рівномірний, нормальний, кластери, решітка, колінеарні, коло) і зерно генератора `seed`; при однаковому зерні набір відтворюється.
3. *Файл*: текстовий файл, кожен рядок якого містить два цілі числа «x y» з діапазону $[0, 10^6]$; порожні рядки й коментарі після символу `#` ігноруються.

Запуск алгоритму. Кнопка «Запустити алгоритм» (або Ctrl+R) обчислює найближчих сусідів для поточного набору точок. Якщо увімкнено прапорець «Перераховувати автоматично», алгоритм запускається також після кожної зміни набору; якщо вимкнено, набір можна спершу повністю ввести, а потім запустити обчислення окремо. Повторний запуск на тому самому наборі дає змогу повторно виміряти час роботи.

Вивід.

- робоча область з шарами, які вмикаються окремо: діаграма Вороного, тріангуляція Делоне, стрілки $p \to \mathrm{NN}(p)$, взаємні пари, точки, номери точок; масштаб колесом миші, зсув перетягуванням, підказка з координатами та сусідом при наведенні;
- панель «Результат»: кількість точок і різних точок, ребер Делоне, трикутників, взаємних пар, мінімальна та максимальна відстань до сусіда, час роботи алгоритму;
- кнопка «Звірити з наївним $O(N^2)$»: запускає повний перебір, порівнює відповіді поелементно та виводить час обох алгоритмів;

Таблиця 3. Модулі та основні функції програми

Функція	Призначення	Складність
`geometry.py`: точка та предикати		
`Point`	незмінна точка з цілими координатами, хешується, порівнюється за (x, y)	—
`dist2(a, b)`	квадрат відстані $d^2(a, b)$	$O(1)$
`ccw(a, b, c)`	орієнтація трійки, формула (1)	$O(1)$
`in_circle(a, b, c, d)`	положення d відносно кола через a, b, c, формула (2)	$O(1)$
`circumcenter(a, b, c)`	центр описаного кола (вершина Вороного)	$O(1)$
`quadedge.py`: структура quad-edge		
`make_edge(a, b)`	нове ізольоване ребро $a \to b$	$O(1)$
`splice(a, b)`	злиття або розрізання кілець onext	$O(1)$
`connect(a, b)`	ребро від a.dest до b.org у спільній грані	$O(1)$
`delete_edge(e)`	від'єднання ребра	$O(1)$
`iter_edges(e)`	обхід усіх ребер, кожне один раз	$O(N)$
`delaunay.py`: тріангуляція		
`_build(pts, lo, hi)`	рекурсія «розділяй і володарюй» та злиття (алгоритм 2)	$O(N \log N)$
`delaunay(points)`	сортування, перевірка дублікатів, запуск `_build`	$O(N \log N)$
`nearest.py`, `brute_force.py`: розв'язок задачі		
`all_nearest_neighbors`	основний алгоритм (алгоритм 1)	$O(N \log N)$
`all_nearest_neighbors_brute`	повний перебір, еталон	$O(N^2)$
`voronoi.py`, `generator.py`, `scene.py`: візуалізація та дані		
`voronoi_edges(e)`	ребра $\mathrm{Vor}(S)$ як двоїсті до ребер $\mathrm{DT}(S)$: відрізки, промені, прямі	$O(N)$
`voronoi_segments`	відсікання ребер прямокутником (алгоритм Ляна–Барські)	$O(N)$
`generate(kind, n, seed)`	генерація n точок одного з шести розподілів	$O(N)$
`compute_scene`	запуск алгоритму, замір часу, підготовка шарів для малювання	$O(N \log N)$

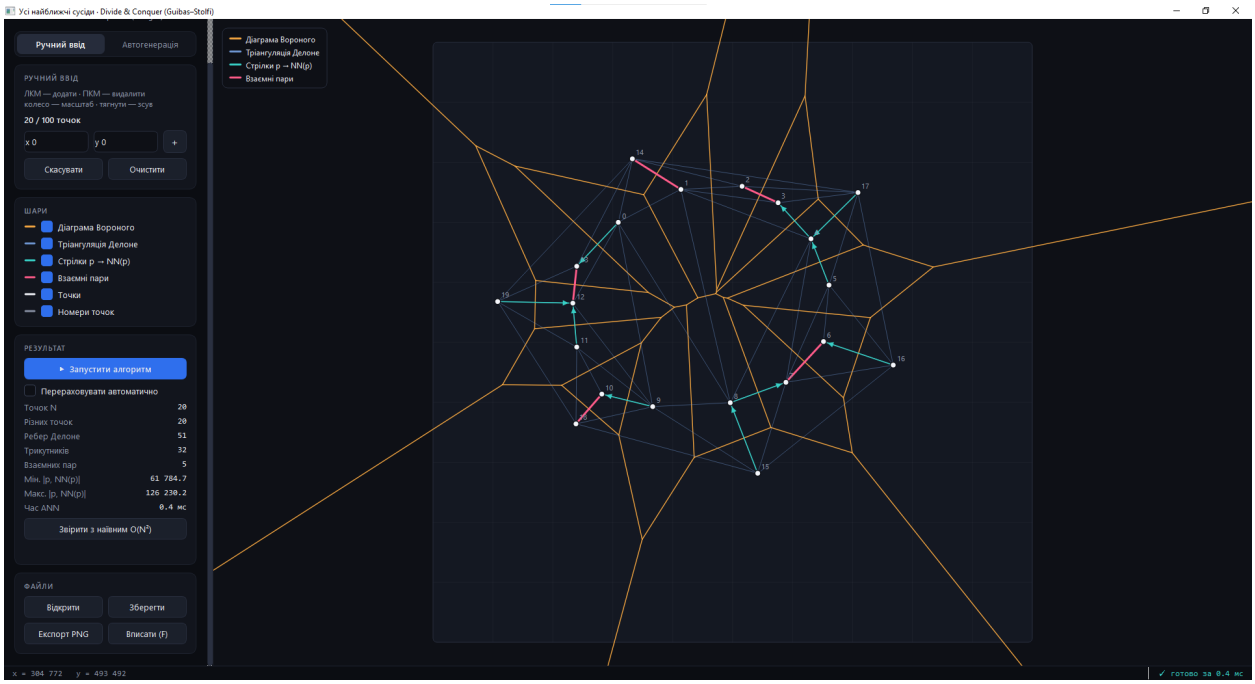


Рис. 5. Головне вікно програми: ліворуч панель вводу, шарів, результату та файлів, праворуч робоча область

- збереження точок у текстовий файл і експорт зображення робочої області у PNG.

4.4. Можливості, програмне та технічне забезпечення

Таблиця 4. Програмне забезпечення

Компонент	Версія та призначення
Python	3.13; мова реалізації
PySide6 (Qt 6)	$\geq$ 6.7; графічний інтерфейс
matplotlib	$\geq$ 3.8; графіки для замірів часу
pytest	$\geq$ 8; автоматичні тести
Операційна система	Windows 10/11 (перевірено); код кросплатформний

Запуск з кореня репозиторію:

```
pip install -r requirements.txt
python ui/app.py                    # графічний інтерфейс
pytest                              # 156 тестів
python benchmarks/benchmark.py      # заміри часу та графіки
```

Спеціального обладнання програма не потребує: усі заміри виконано на звичайному ноутбуці. На рис. 5 показано головне вікно програми.

4.5. Демонстрація на великих і найгірших вхідних даних

Згідно з вимогами програма має демонструвати ефективність для найгіршого вводу. Для оцінки взято три набори по $N = 10^4$ точок (рис. 6 і 7):

- *рівномірний розподіл* є типовим випадком;
- *квадратна решітка* 100×100 дає максимальну кількість коциклічних четвірок: кожна чарунка решітки має чотири точки на одному колі, тож $\text{in_circle} = 0$ і тріангуляція неєдина, а кожна точка має до чотирьох рівновіддалених сусідів;
- *колінеарні точки*: тріангуляція вироджується в ланцюжок без жодного трикутника, а діаграма Вороного складається з паралельних прямих.

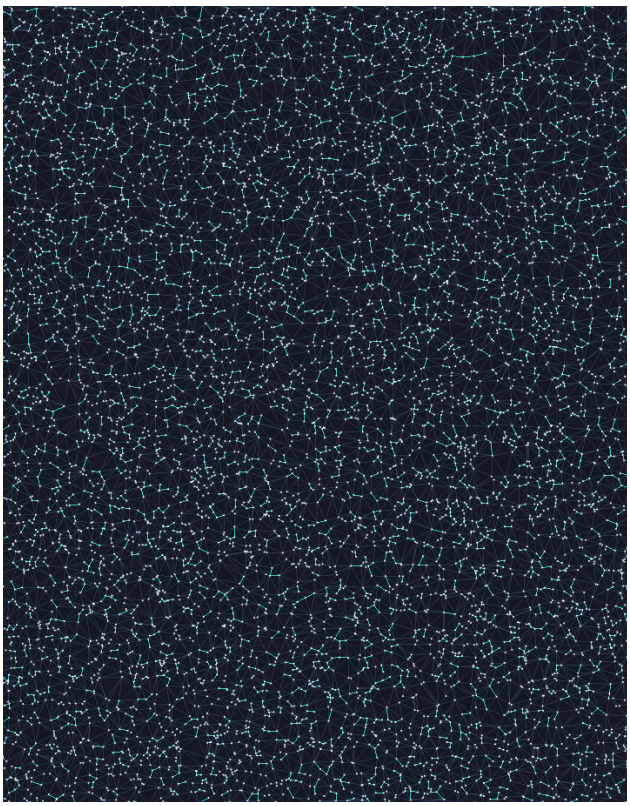
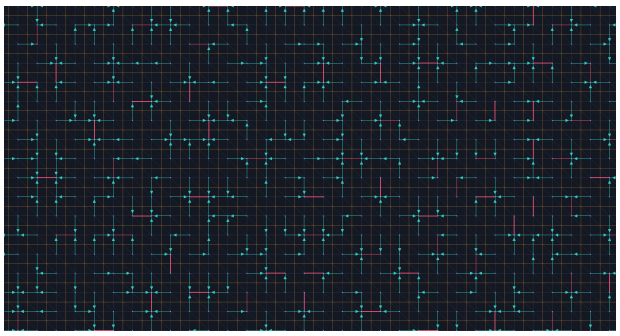
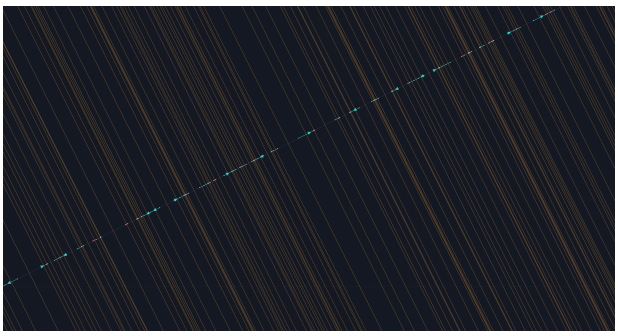


Рис. 6. Рівномірний розподіл, $N = 10^4$: тріангуляція Делоне і стрілки до найближчих сусідів



(а) решітка (фрагмент)



(б) колінеарні точки (фрагмент)

Рис. 7. Найгірші та вироджені випадки, $N = 10^4$, наближені фрагменти: помаранчеві лінії є ребрами Вороного, бірюзові стрілки ведуть до найближчого сусіда, рожеві відрізки позначають взаємні пари

Таблиця 5. Показники з панелі «Результат» для $N = 10^4$

Набір	ребер DT	трикутників	взаємних пар	$\min d$	час, мс
рівномірний	29 977	19 978	3 073	60,6	619,0
решітка 100×100	29 601	19 602	1 256	10 101,0	354,2
колінеарні	9 999	0	3 292	2,2	594,7

Дані таблиці 5 узгоджуються з теорією. Для рівномірного набору $E = 29\,977 \le 3N - 6 = 29\,994$ і трикутників $19\,978 \le 2N - 5$. Для решітки всі відстані до сусіда однакові ($\min d = \max d$ дорівнює кроку решітки), а сусідом обирається точка з меншим індексом, як і в повному переборі. Для колінеарних точок ребер рівно $N - 1$, трикутників немає. Час в інтерфейсі вимірюється разом із накладними витратами фонового потоку й відмалювання, тому він більший за «чисті» заміри з таблиці 2, але для всіх трьох наборів становить менше секунди. Для ручного набору з 20 точок звірка з повним перебором показала збіг для всіх точок.

4.6. Лістинг основних модулів

Нижче наведено ключову функцію, яка розв'язує задачу; повні лістинги модулів ядра подано в додатках А і Б.

Лістинг 1. Функція `all_nearest_neighbors` (`src/nearest.py`)

```
20 def all_nearest_neighbors(points: Sequence[Point]) -> list[Optional[int]]:
21     """Для кожної точки повертає індекс її найближчого сусіда, за O(N log N).
22     Інтерфейс і результат такі самі, як у brute_force.all_nearest_neighbors_brute:
23     result[i] = j, j != i; серед рівновіддалених сусідів - менший індекс;
24     дублікати - сусіди одне одного; для N = 1 результат [None].
25     """
26     n = len(points)
27     result: list[Optional[int]] = [None] * n
28     if n < 2:
29         return result
```

```
30
31      # ---------- 1. Дублікати, O(N) ----------
32      # groups[p] - індекси всіх входжень точки p у порядку зростання
33      groups: dict[Point, list[int]] = {}
34      for i, p in enumerate(points):
35          groups.setdefault(p, []).append(i)
36
37      for idx in groups.values():
38          if len(idx) > 1:
39              # найближчий сусід - інша копія на відстані 0, з найменшим індексом
40              first, second = idx[0], idx[1]
41              for i in idx:
42                  result[i] = second if i == first else first
43
44      unique = list(groups)
45      if len(unique) == 1:
46          return result
47
48      # представник точки - її перше входження (найменший індекс)
49      rep = {p: idx[0] for p, idx in groups.items()}
50
51      # ---------- 2. Тріангуляція Делоне, O(N log N) ----------
52      start = delaunay(unique)
53
54      # ---------- 3. Найближчий серед сусідів у тріангуляції, O(N) ----------
55      # best[p] = (квадрат відстані, індекс сусіда)
56      best: dict[Point, tuple[int, int]] = {}
57      for e in iter_edges(start):
58          for a, b in ((e.org, e.dest), (e.dest, e.org)):
59              cand = (dist2(a, b), rep[b])
60              if a not in best or cand < best[a]:
61                  best[a] = cand
62
63      for p, idx in groups.items():
64          if len(idx) == 1:
65              result[idx[0]] = best[p][1]
66
67      return result
```

5. Висновки

У роботі розв'язано задачу «Усі найближчі сусіди» для N точок площини методом «розділяй і володарюй» з оптимальною складністю $\Theta(N \log N)$.

Науковий аналіз виконаної роботи. Задачу зведено до побудови тріангуляції Делоне. Доведено, що ребро до кожного найближчого сусіда, включно з усіма рівновіддаленими, належить будь-якій тріангуляції Делоне (теорема 1). Звідси випливає, що після побудови тріангуляції відповідь знаходиться за $O(N)$ і не залежить від

вибору тріангуляції у вироджених випадках. Тріангуляцію будує алгоритм Гібаса–Столфі; покроково показано, що злиття половин лінійне, бо кожне ребро додається і видаляється не більше одного разу, а рекурентність $T(N) = 2T(N/2) + O(N)$ дає $O(N \log N)$. Ця оцінка збігається з нижньою межею $\Omega(N \log N)$ для задачі, тож алгоритм оптимальний за порядком. Порівняно з побудовою діаграми Вороного через розділяючий ланцюг, обраний варіант зводить усю геометрію до двох точних цілочисельних предикатів і не потребує обчислення перетинів.

Експерименти узгоджуються з теорією. Нахил залежності час–N у логарифмічних шкалах становить 1,10 проти 2,03 для повного перебору. Нормований час майже сталий на всіх шести розподілах, а для $N = 4000$ алгоритм швидший за перебір в 11,5 раза. Вироджені дані (решітка з коциклічними точками, колінеарні точки, точки біля кола) обробляються коректно і не сповільнюють роботу. Коректність підтверджено 156 автоматичними тестами та поелементною звіркою з повним перебором.

Створено програму з графічним інтерфейсом, яка підтримує ручний ввід до 100 точок, автоматичну генерацію до $2 \cdot 10^4$ точок шести розподілів, роботу з файлами, пошарову візуалізацію тріангуляції Делоне, діаграми Вороного та пар сусідів, а також звірку з повним перебором.

Проблеми та обмеження.

- Реалізація мовою Python має великий сталий множник: близько 4 мкс на одиницю $N \log_2 N$. Для 10^6 точок це десятки секунд, хоча асимптотика правильна.
- Точна арифметика спирається на цілі координати. Для дійсних координат потрібні або адаптивні точні предикати, або масштабування до цілої сітки.
- Структура quad-edge з окремим об'єктом на кожне чверть-ребро займає значно більше пам'яті, ніж компактні масивові подання тріангуляції.
- Алгоритм послідовний; крок злиття за своєю природою послідовний, тож пряме розпаралелювання ускладнене.

Перспективи та умови покращення ефективності.

- Перенесення ядра на C++ з 128-бітними цілими або адаптивною арифметикою зменшить сталий множник на один-два порядки при тій самій асимптотиці.
- Поділ почергово за x і y (ідея Двайєра) зменшує очікувану кількість роботи на етапі злиття для рівномірно розподілених точок.
- Рекурсивні виклики для двох половин незалежні, тому їх можна виконувати паралельно на верхніх рівнях рекурсії; сучасні паралельні методи побудови діаграм Вороного та k-d-дерев [6–10] показують, як масштабувати такі обчислення на багатоядерні системи та графічні процесори.
- Тріангуляцію Делоне можна використати й для споріднених задач без зміни складності: найближча пара (найкоротше ребро), евклідове мінімальне кістякове дерево, найбільше порожнє коло [13]. Узагальнення на k найближ-

чих сусідів потребує інших структур, наприклад діаграм Вороного вищих порядків або просторових дерев.

Список літератури

[1] Ebadian S., Zarrabi-Zadeh H. A simple randomized algorithm for all nearest neighbors. Proceedings of the 31st Canadian Conference on Computational Geometry (CCCG 2019), Edmonton, Canada, 2019, pp. 94–98.
https://cccg.ca/proceedings/2019/proceedings.pdf

[2] Chan T. M. Dynamic geometric data structures via shallow cuttings. 35th International Symposium on Computational Geometry (SoCG 2019), LIPIcs, vol. 129, Portland, USA, 2019, pp. 24:1–24:13. DOI: 10.4230/LIPIcs.SoCG.2019.24

[3] Chan T. M., Rahul S., Xue J. Range closest-pair search in higher dimensions. Computational Geometry: Theory and Applications. Vol. 91, 2020, art. 101669. arXiv:1905.01029

[4] Biniaz A., Bose P., Chung C., De Carufel J.-L., Iacono J., Maheshwari A., Odak S., Smid M., Tóth C. D. Tight bounds on the number of closest pairs in vertical slabs. 19th Algorithms and Data Structures Symposium (WADS 2025), LIPIcs, vol. 349, Toronto, Canada, 2025, pp. 8:1–8:14. DOI: 10.4230/LIPIcs.WADS.2025.8

[5] Williams R. On the difference between closest, furthest, and orthogonal pairs: nearly-linear vs barely-subquadratic complexity in computational geometry. Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018), New Orleans, USA, 2018, pp. 1207–1215. arXiv:1709.05282

[6] Blelloch G. E., Dobson M. Parallel nearest neighbors in low dimensions with batch updates. Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2022), Alexandria, USA (online), 2022, pp. 195–208. arXiv:2111.04182

[7] Men Z., Shen Z., Gu Y., Sun Y. Parallel kd-tree with batch updates. Proceedings of the ACM on Management of Data (SIGMOD 2025), Berlin, Germany. Vol. 3, No. 1, 2025, art. 62, pp. 1–26. arXiv:2411.09275

[8] Mizrachi M., Raveh B., Steinberg E. MadVoro: parallel construction of Voronoi diagrams in distributed memory systems. RAS Techniques and Instruments, 2025.
https://arxiv.org/abs/2502.14825

[9] Xiao Y., Li Y., Cao J., Chen Z. Efficient computation of Voronoi diagrams using point-in-cell tests. Препринт, 2025.
https://arxiv.org/abs/2509.07175

[10] Byrohl C., Nelson D., Singh S. D. votess: a multi-target, GPU-capable, parallel Voronoi tessellator. Препринт, 2024.
https://arxiv.org/abs/2412.04514

[11] Preparata F. P., Shamos M. I. Computational Geometry: An Introduction. Springer-Verlag, New York, 1985. 398 p.

[12] Терещенко В. М. Аналіз методів розв'язання оптимізаційних задач обчислювальної геометрії: навчальний посібник. ВПЦ «Київський університет», Київ, 2022. 111 с.

[13] Терещенко В. М. Обчислювальна геометрія: конспект лекцій (лекції 2.2, 9, 10). Київський національний університет імені Тараса Шевченка, Київ, 2025.

[14] Guibas L., Stolfi J. Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams. ACM Transactions on Graphics. Vol. 4, No. 2, 1985, pp. 74–123.

Додатки

Повні лістинги двох ключових модулів: побудови тріангуляції Делоне та пошуку сусідів. Решта коду (предикати, quad-edge, інтерфейс, генератор, візуалізація, тести, заміри) доступна в репозиторії `https://github.com/kyrylobatrak/all-nearest-neighbors`.

A. Модуль `delaunay.py`

```
"""Побудова тріангуляції Делоне методом «розділяй і володарюй» (Guibas & Stolfi, 1985).

Рекурсія:
    1. точки відсортовано за (x, y);
    2. множина ділиться навпіл за медіаною;
    3. тріангуляції половин будуються рекурсивно;
    4. злиття: знаходиться нижня спільна дотична, далі половини
       «зшиваються» знизу вгору; ребра, що порушують умову Делоне
       (порожнього кола), видаляються.

Складність: T(N) = 2T(N/2) + O(N) = O(N log N).
"""

from __future__ import annotations
from typing import Sequence
from geometry import Point, ccw, in_circle
from quadedge import Edge, connect, delete_edge, make_edge, splice

def _left_of(p: Point, e: Edge) -> bool:
    """Точка p строго зліва від напрямленого ребра e."""
    return ccw(p, e.org, e.dest) > 0

def _right_of(p: Point, e: Edge) -> bool:
    """Точка p строго справа від напрямленого ребра e."""
    return ccw(p, e.dest, e.org) > 0

def _build(pts: Sequence[Point], lo: int, hi: int) -> tuple[Edge, Edge]:
    """Тріангуляція Делоне точок pts[lo:hi] (відсортованих, без дублікатів, щонайменше 2).
    Повертає (ldo, rdo):
        ldo - ребро опуклої оболонки проти год. стрілки, що виходить з найлівішої точки;
        rdo - ребро опуклої оболонки за год. стрілкою, що виходить з найправішої точки.
    """
    n = hi - lo
    # ---------- база рекурсії ----------
    if n == 2:
```

```
36          a = make_edge(pts[lo], pts[lo + 1])
37          return a, a.sym
38
39      if n == 3:
40          p1, p2, p3 = pts[lo], pts[lo + 1], pts[lo + 2]
41          a = make_edge(p1, p2)
42          b = make_edge(p2, p3)
43          splice(a.sym, b)
44          o = ccw(p1, p2, p3)
45          if o > 0:                       # лівий поворот: трикутник p1 p2 p3
46              connect(b, a)
47              return a, b.sym
48          if o < 0:                       # правий поворот: трикутник p1 p3 p2
49              c = connect(b, a)
50              return c.sym, c
51          return a, b.sym                 # колінеарні: ланцюжок з двох ребер
52
53      # ---------- розділення ----------
54      mid = (lo + hi) // 2
55      ldo, ldi = _build(pts, lo, mid)
56      rdi, rdo = _build(pts, mid, hi)
57
58      # ---------- злиття, крок 1: нижня спільна дотична ----------
59      # ldi рухається по оболонці лівої частини, rdi - правої,
60      # доки відрізок між ними не залишить обидві оболонки над собою.
61      while True:
62          if _left_of(rdi.org, ldi):
63              ldi = ldi.lnext
64          elif _right_of(ldi.org, rdi):
65              rdi = rdi.rprev
66          else:
67              break
68
69      # базове ребро: від правої частини до лівої
70      basel = connect(rdi.sym, ldi)
71      if ldi.org == ldo.org:
72          ldo = basel.sym
73      if rdi.org == rdo.org:
74          rdo = basel
75
76      # ---------- злиття, крок 2: «застібка» знизу вгору ----------
77      def valid(e: Edge) -> bool:
78          """Кандидат лежить над базовим ребром."""
79          return _right_of(e.dest, basel)
80
81      while True:
82          # кандидат з лівої частини; видаляємо ребра, що порушують умову Делоне
83          lcand = basel.sym.onext
```

```
        if valid(lcand):
            while in_circle(basel.dest, basel.org, lcand.dest, lcand.onext.dest) > 0:
                t = lcand.onext
                delete_edge(lcand)
                lcand = t

        # кандидат з правої частини
        rcand = basel.oprev
        if valid(rcand):
            while in_circle(basel.dest, basel.org, rcand.dest, rcand.oprev.dest) > 0:
                t = rcand.oprev
                delete_edge(rcand)
                rcand = t

        lv, rv = valid(lcand), valid(rcand)
        if not lv and not rv:           # дійшли до верхньої спільної дотичної
            break

        # нове ребро «застібки»: обираємо кандидата, чиє коло порожнє
        if not lv or (rv and in_circle(lcand.dest, lcand.org, rcand.org, rcand.dest)
        ↪ > 0):
            basel = connect(rcand, basel.sym)
        else:
            basel = connect(basel.sym, lcand.sym)

    return ldo, rdo

def delaunay(points: Sequence[Point]) -> Edge:
    """Тріангуляція Делоне множини різних точок (щонайменше 2).
    Повертає будь-яке ребро результату; решту можна обійти через
    quadedge.iter_edges. Точки сортуються всередині.
    """
    pts = sorted(points)
    if len(pts) < 2:
        raise ValueError("потрібно щонайменше 2 точки")
    for a, b in zip(pts, pts[1:]):
        if a == b:
            raise ValueError(f"дублікат точки {a}; дублікати треба прибрати
            ↪ заздалегідь")
    ldo, _ = _build(pts, 0, len(pts))
    return ldo
```

Б. Модуль `nearest.py`

```
"""Усі найближчі сусіди через тріангуляцію Делоне.

Теорема. Якщо q - найближчий сусід p, то pq - ребро будь-якої тріангуляції
Делоне множини S.
```

```
Доведення: у крузі з діаметром pq будь-яка точка r != q (включно з межею)
задовольняє |pr| < |pq|, тобто була б ближчою до p. Отже круг порожній,
pq - ребро графа Габріеля, а він є підграфом тріангуляції Делоне.

Тому для кожної точки досить переглянути лише її сусідів у тріангуляції.
Сумарно це Σ deg(p) = 2E <= 2(3N - 6) порівнянь, тобто O(N).
"""

from __future__ import annotations
from typing import Optional, Sequence
from delaunay import delaunay
from geometry import Point, dist2
from quadedge import iter_edges


def all_nearest_neighbors(points: Sequence[Point]) -> list[Optional[int]]:
    """Для кожної точки повертає індекс її найближчого сусіда, за O(N log N).
    Інтерфейс і результат такі самі, як у brute_force.all_nearest_neighbors_brute:
    result[i] = j, j != i; серед рівновіддалених сусідів - менший індекс;
    дублікати - сусіди одне одного; для N = 1 результат [None].
    """
    n = len(points)
    result: list[Optional[int]] = [None] * n
    if n < 2:
        return result

    # ---------- 1. Дублікати, O(N) ----------
    # groups[p] - індекси всіх входжень точки p у порядку зростання
    groups: dict[Point, list[int]] = {}
    for i, p in enumerate(points):
        groups.setdefault(p, []).append(i)

    for idx in groups.values():
        if len(idx) > 1:
            # найближчий сусід - інша копія на відстані 0, з найменшим індексом
            first, second = idx[0], idx[1]
            for i in idx:
                result[i] = second if i == first else first

    unique = list(groups)
    if len(unique) == 1:
        return result

    # представник точки - її перше входження (найменший індекс)
    rep = {p: idx[0] for p, idx in groups.items()}

    # ---------- 2. Тріангуляція Делоне, O(N log N) ----------
    start = delaunay(unique)
```

```
53
54      # ---------- 3. Найближчий серед сусідів у тріангуляції, O(N) ----------
55      # best[p] = (квадрат відстані, індекс сусіда)
56      best: dict[Point, tuple[int, int]] = {}
57      for e in iter_edges(start):
58          for a, b in ((e.org, e.dest), (e.dest, e.org)):
59              cand = (dist2(a, b), rep[b])
60              if a not in best or cand < best[a]:
61                  best[a] = cand
62
63      for p, idx in groups.items():
64          if len(idx) == 1:
65              result[idx[0]] = best[p][1]
66
67      return result
```